Compositional Constructive Metamorphism for Testing Constraint Checking Implementations (Appendix)

Anonymous Authors

I. Semantics of Constraint Checking

To give the proof of the theorem, we first give a comprehensive explanation of Constraint Checking (CC). For a given input (P, S), a CC implementation will return a truth value T and a link set L as evaluation result. To ease discussions, we assume that every variable in the input constraint S is used by one quantifier, e.g., constraints like $(\forall x \in (S_1))$ and $(\forall x \in (S_2))$ will not be considered because it is equal to $(\forall x \in (S_1))$ and $(\forall y \in (S_2[y/x]))$. We will formally define the evaluation rules of the truth value and link set.

First, we define some concepts that will be used in our further discussion of evaluation rules. We define the assignment set that contains variable bindings which have already been applies by quantifiers during the evaluation, formally: $\alpha = \{v_1 \mapsto e_1, v_2 \mapsto e_2, \dots\}$ where $v_i \mapsto e_i$ represents variable v_i is bound to element e_i. We define the truth value evaluation as a function $\mathcal{T}[S]_\alpha$ and the link set generation as a function $\mathcal{L}[S]_\alpha$, while α can be omitted if empty. The context pool P is implicitly referenced in the evaluation of both functions, as the evaluation of S may involve accessing elements from the context sets defined in P.

A. The truth value evaluation

The truth value T is a Boolean value that indicates whether the constraint S is satisfied (T) or violated (F) against the context pool P. In the following, we list CC's truth value evaluation rules ($\mathcal{T}[S]_\alpha$) by the categories of the operators.

1) Quantifiers, i.e., $\forall$, $\exists$: The detailed truth evaluation rules for quantifiers in CC are defined as follows, which are similar to the evaluation of first-order-logic (FOL) formulas:

$$\begin{aligned}&\mathcal{T}[\forall v \in C(S)]_\alpha = \\ &\mathsf{T} \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_1\}} \wedge \dots \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_n\}} \mid e_i \in C.\end{aligned} \tag{1}$$

$$\begin{aligned}&\mathcal{T}[\exists v \in C(S)]_\alpha = \\ &\mathsf{F} \vee \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_1\}} \vee \dots \vee \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_n\}} \mid e_i \in C.\end{aligned} \tag{2}$$

2) Connectives, i.e., and, or, implies*:* The evaluation rules for logical connectives naturally follow standard Boolean algebra. They evaluate the truth value of compound constraints by recursively evaluating their sub-constraints:

$$\mathcal{T}[(S_1) \text{ and } (S_2)]_\alpha = \mathcal{T}[S_1]_\alpha \wedge \mathcal{T}[S_2]_\alpha. \tag{3}$$

$$\mathcal{T}[(S_1) \text{ or } (S_2)]_\alpha = \mathcal{T}[S_1]_\alpha \vee \mathcal{T}[S_2]_\alpha. \tag{4}$$

$$\mathcal{T}[(S_1) \text{ implies } (S_2)]_\alpha = \neg\mathcal{T}[S_1]_\alpha \vee \mathcal{T}[S_2]_\alpha. \tag{5}$$

3) Negation, i.e., not*:* The negation operator simply inverts the truth value of the underlying constraint, which is evaluated as:

$$\mathcal{T}[\text{not } (S)]_\alpha = \neg\mathcal{T}[S]_\alpha. \tag{6}$$

4) Bfunctions, i.e., $bf(\dots)$: The evaluation of a bfunction $bf(\dots)$ is determined by the specific semantics of the bfunction:

$$\mathcal{T}[bf(\dots)]_\alpha = \text{the evaluation result of } bf(\dots) \text{ under the variable bindin} \tag{7}$$

B. The link set generation

The link set L contains links that explains how a specific consistency constraints is satisfied or violated via listing the certain related elements. A link is a set of variable bindings: $l = \{v_1 \mapsto e_1, v_2 \mapsto e_2, \dots\}$ which means concurrently binding $v_i \mapsto e_i$ causes the satisfaction or violation.

To illustrate the link set generation rules of CC, we first define some operators between link sets which has already been mentioned in Sec. III.D.

Union. The union ($\cup$) between link sets is same as the basic set operations. It is used for expressing a alternative relationship (e.g., satisfying an or only need to satisfy any of its operands). The generalized big union ($\bigcup$) extends this operation to a family of link sets, aggregating all variable bindings from multiple sets into a single comprehensive set.

Cartesian Product. The cartesian product ($\otimes$) concatenates two link sets by union the variable bindings to the link pairs combined with every link in set L_1 and every link in set L_2, i.e., $L_1 \otimes L_2 = \{l_1 \cup l_2 \mid l_1 \in L_1 \wedge l_2 \in L_2\}$, if $L_1 \neq \emptyset$ and $L_2 \neq \emptyset$; otherwise, $L_1 \cup L_2$.

We then list the link set generation rules ($\mathcal{L}[S]_\alpha$) by categories of the operators.

1) Quantifiers, i.e., $\forall$, $\exists$: For quantifiers, the link set captures the specific element bindings that determine the overall truth value. A universal quantifier ($\forall$) is violated if any single element fails the constraint, while an existential quantifier ($\exists$) is satisfied if any single element meets it. The rules aggregate these critical bindings accordingly:

$$\mathcal{L}[\forall v \in C(S)]_\alpha = \bigcup_{e_i \in C \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_i\}} = \mathsf{F}} (\{\{v \mapsto e_i\}\} \otimes \mathcal{L}[S]_{\alpha \cup \{v \mapsto e_i\}}). \quad (8)$$

$$\mathcal{L}[\exists v \in C(S)]_\alpha = \bigcup_{e_i \in C \wedge \mathcal{T}[S]_{\alpha \cup \{v \mapsto e_i\}} = \mathsf{T}} (\{\{v \mapsto e_i\}\} \otimes \mathcal{L}[S]_{\alpha \cup \{v \mapsto e_i\}}). \quad (9)$$

Note that $\forall$ does not generate links for elements that satisfy the constraint because they do not contribute to the violation, and $\exists$ does not generate links for elements that violate the constraint because they do not contribute to the satisfaction. Only the critical bindings that directly lead to the overall truth value are captured in the link set.

2) Connectives, i.e., $\mathsf{and}, \mathsf{or}, \mathsf{implies}$*:* Logical connectives build their link sets based on how their operands contribute to the truth value. When a connective requires concurrent conditions (e.g., both operands of an and being true, or both operands of an or being false), we use the Cartesian product ($\otimes$). When a connective allows alternative conditions, we use the union ($\bigcup$).

$$\mathcal{L}[(S_1) \text{ and } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \\ \bigcup_{S_i \in \{S_1, S_2\} \wedge \mathcal{T}[S_i]_\alpha = \mathsf{F}} \mathcal{L}[S_i]_\alpha, & \text{otherwise} \end{cases}. \quad (10)$$

$$\mathcal{L}[(S_1) \text{ or } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \bigcup_{S_i \in \{S_1, S_2\} \wedge \mathcal{T}[S_i]_\alpha = \mathsf{T}} \mathcal{L}[S_i]_\alpha, & \text{otherwise} \end{cases}. \quad (11)$$

$$\mathcal{L}[(S_1) \text{ implies } (S_2)]_\alpha = \begin{cases} \mathcal{L}[S_1]_\alpha \otimes \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \mathcal{L}[S_1]_\alpha \cup \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \\ \mathcal{L}[S_1]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{F} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{F} \\ \mathcal{L}[S_2]_\alpha, & \text{if } \mathcal{T}[S_1]_\alpha = \mathsf{T} \wedge \mathcal{T}[S_2]_\alpha = \mathsf{T} \end{cases}. \quad (12)$$

It is worth noting that the implication (S_1) implies (S_2) is logically equivalent to $(\text{not } (S_1))$ or (S_2). While its link set generation could technically be formulated using the $\bigcup$ operator similar to the preceding rules, we explicitly expand it into four distinct cases. This formulation is deliberately chosen for better readability and expressional clarity.

3) Negation, i.e., not*:* The negation operator simply inverts the truth value but preserves the exact same triggering reasons (links) of its underlying operand. Therefore, it directly inherits the link set of its sub-constraint.

Finally, we formulate a CC implementation as I_{CC}. If the implementation is correct, we have $I_{CC}(P, S) = \langle \mathcal{T}[S], \mathcal{L}[S] \rangle$.

$$\mathcal{L}[\text{not } (S)]_\alpha = \mathcal{L}[S]_\alpha. \quad (13)$$

4) Bfunctions, i.e., $bf(\dots)$: A bfunction does not generate any link because all variable bindings that contribute to its truth value are already captured by the link sets of the quantifiers that bind these variables. Therefore, we have:

$$\mathcal{L}[bf(\dots)]_\alpha = \emptyset. \quad (14)$$

II. Definition of A Precise Primitive Unit

After introducing the semantics of CC, we now move to the proof of the soundness of CCM. The key intermediate artifact of CCM is a *primitive unit* (PU), defined as a triple $u = \langle s, p, t \rangle$, where s is a (sub-)constraint formula, p is an associated context pool, and t is a *semantic tag*. The tag summarizes the semantic expectations of evaluating s under p for a correct CC implementation. Therefore, to establish soundness, we first formalize when a PU tag is *precise*.

We distinguish two kinds of PUs. A PU is *closed* if s contains no free variables and can be directly evaluated. A PU is *open* if s contains free variables and cannot be evaluated without additional bindings. Following our methodology, we restrict each open PU to contain *at most one* free variable, denoted by x.

Definition 1 (Precise Closed Units)

A closed unit $u = \langle s, p, t \rangle$ with tag $t = \langle T, L \rangle$ is *precise* if and only if, for a correct CC implementation I_{CC}, we have

$$I_{CC}(p, s) = \langle T, L \rangle.$$

Equivalently, under the formal semantics in Sec. I, $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$ (where the context sets referenced by s are obtained from the pool p).

Definition 2 (Precise Open Units)

An open unit $u = \langle s, p, t \rangle$ whose constraint s has *at most one* free variable (denoted by x), with tag $t = \langle x, C_\mathsf{T}, C_\mathsf{F} \rangle$, is *precise* if and only if C_T and C_F are disjoint sets of elements such that binding x to any element in C_T satisfies s, while binding x to any element in C_F violates s, i.e.,

$$(C_\mathsf{T} \cap C_\mathsf{F} = \emptyset) \wedge (\forall e \in C_\mathsf{T},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}) \wedge (\forall e \in C_\mathsf{F},\ \mathcal{T}[s]_{\{x \mapsto e\}} =$$

Definition 3 (Precise Primitive Units)

A unit $u = \langle s, p, t \rangle$ is *precise* if and only if it is a precise closed unit or a precise open unit.

III. Lemmas and Their Proofs

A. Soundness of Step I

Before proving the soundness of Step I, we introduce two auxiliary functions used by CCM to infer PU tags from a source execution result. These functions are purely structural operators over link sets and will be used as building blocks in Lemma 1.

Let $\mathsf{Vars}(s)$ denote the set of variables that syntactically occur in a constraint formula s. For a link l (a set of bindings), and a variable set V, we define the *restriction* of l to V as $l\restriction_V = \{v \mapsto e \in l \mid v \in V\}$.

Definition 4 (Attribution Function Attr)

Given a link set L and a sub-constraint formula s, the attribution function projects each link to the variables relevant to s:

$$\mathsf{Attr}(L, s) = \{\, l\restriction_{\mathsf{Vars}(s)} \mid l \in L \wedge l\restriction_{\mathsf{Vars}(s)} \neq \emptyset \,\}.$$

Intuitively, Attr removes bindings irrelevant to s while preserving evidence about which element bindings contributed to the source evaluation.

Definition 5 (Extraction Function Extract)

Given a link set L and a variable x, the extraction function collects all elements that are bound to x in any link:

$$\mathsf{Extract}(L, x) = \{\, e \mid \exists l \in L,\ (x \mapsto e) \in l \,\}.$$

In Step I, Extract is used when an open PU is obtained by removing a quantifier $Qx \in C(\cdot)$ and making x free: the extracted elements form the witness (or counterexample) set for x, and the remaining elements of the quantifier domain C can be conservatively assigned to the opposite context. (When $L = \emptyset$, we have $\mathsf{Extract}(L, x) = \emptyset$.)

Lemma 1 (Soundness of Step I)

Given a source test case and a correct CC evaluation result of it, the initial PUs CCM extracted from the case are always precise.

Proof. Let the source test case be $(P_{\text{src}}, S_{\text{src}})$. Since the CC implementation under test is assumed correct, its source execution result equals the formal semantics:

$$I_{CC}(P_{\text{src}}, S_{\text{src}}) = \langle T_{\text{src}}, L_{\text{src}} \rangle = \langle \mathcal{T}[S_{\text{src}}], \mathcal{L}[S_{\text{src}}] \rangle.$$

(1) Decomposition-based extraction for connectives

Assume S_{src} is rooted at a binary connective, i.e., $S_{\text{src}} = (S_1)\ \mathsf{op}\ (S_2)$ where $\mathsf{op} \in \{\mathsf{and}, \mathsf{or}, \mathsf{implies}\}$. Step I constructs candidate closed PUs $u_i = \langle S_i, P_{\text{src}}, \langle T_i, L_i \rangle \rangle$, where $L_i = \mathsf{Attr}(L_{\text{src}}, S_i)$, and T_i is either deduced from T_{src} when unambiguous, or inferred from the non-emptiness of L_i when the branch truth is ambiguous. We prove that whenever such a PU is produced, it is a *precise closed unit*, i.e., $T_i = \mathcal{T}[S_i]$ and $L_i = \mathcal{L}[S_i]$.

We give the argument in detail for $\mathsf{op} = \mathsf{and}$; the other two connectives follow analogously from their corresponding rules in Eqs. (4)–(5) and Eqs. (11)–(12).

For $S_{\text{src}} = (S_1)$ and (S_2), by Eq. (3),

$$T_{\text{src}} = \mathcal{T}[S_{\text{src}}] = \mathcal{T}[S_1] \wedge \mathcal{T}[S_2].$$

Case 1 (truth-determinable). If $T_{\text{src}} = \mathsf{T}$, then necessarily $\mathcal{T}[S_1] = \mathsf{T}$ and $\mathcal{T}[S_2] = \mathsf{T}$; hence setting $T_i = \mathsf{T}$ is correct. Moreover, by Eq. (10) (the T branch),

$$L_{\text{src}} = \mathcal{L}[S_{\text{src}}] = \mathcal{L}[S_1] \otimes \mathcal{L}[S_2].$$

Under our normal-form assumption that bound variables are uniquely named (alpha-renaming if needed), $\mathsf{Vars}(S_1)$ and $\mathsf{Vars}(S_2)$ are disjoint. Therefore, restricting any link $l_1 \cup l_2 \in \mathcal{L}[S_1] \otimes \mathcal{L}[S_2]$ to $\mathsf{Vars}(S_1)$ yields exactly l_1. By Definition 4, $L_1 = \mathsf{Attr}(L_{\text{src}}, S_1) = \mathcal{L}[S_1]$; symmetrically $L_2 = \mathcal{L}[S_2]$. Consequently, for each produced u_i, its tag satisfies $\langle T_i, L_i \rangle = \langle \mathcal{T}[S_i], \mathcal{L}[S_i] \rangle$, and thus u_i is precise by Definition 1.

More formally, let $V_1 = \mathsf{Vars}(S_1)$ and $V_2 = \mathsf{Vars}(S_2)$ with $V_1 \cap V_2 = \emptyset$. We first prove $\mathsf{Attr}(L_{\text{src}}, S_1) \subseteq \mathcal{L}[S_1]$. Take any $l' \in \mathsf{Attr}(L_{\text{src}}, S_1)$. By Definition 4, there exists some $l \in L_{\text{src}}$ such that $l' = l\restriction_{V_1} \neq \emptyset$. Since $L_{\text{src}} = \mathcal{L}[S_1] \otimes \mathcal{L}[S_2]$, by the definition of $\otimes$ either (i) $l \in \mathcal{L}[S_1]$ (when $\mathcal{L}[S_2] = \emptyset$), or (ii) $l = l_1 \cup l_2$ for some $l_1 \in \mathcal{L}[S_1]$ and $l_2 \in \mathcal{L}[S_2]$ (when both are non-empty). In both cases, using that links in $\mathcal{L}[S_2]$ only bind variables from V_2, we have $l\restriction_{V_1} = l_1 \in \mathcal{L}[S_1]$, hence $l' \in \mathcal{L}[S_1]$.

Next we prove $\mathcal{L}[S_1] \subseteq \mathsf{Attr}(L_{\text{src}}, S_1)$. Take any $l_1 \in \mathcal{L}[S_1]$. If $\mathcal{L}[S_2] = \emptyset$, then $L_{\text{src}} = \mathcal{L}[S_1] \cup \mathcal{L}[S_2] = \mathcal{L}[S_1]$, so $l_1 \in L_{\text{src}}$ and $l_1 = l_1\restriction_{V_1} \in \mathsf{Attr}(L_{\text{src}}, S_1)$. Otherwise, pick any $l_2 \in \mathcal{L}[S_2]$ and let $l = l_1 \cup l_2 \in \mathcal{L}[S_1] \otimes \mathcal{L}[S_2] = L_{\text{src}}$; then $l\restriction_{V_1} = l_1$, so again $l_1 \in \mathsf{Attr}(L_{\text{src}}, S_1)$. Therefore $\mathsf{Attr}(L_{\text{src}}, S_1) = \mathcal{L}[S_1]$ (and symmetrically for S_2).

Case 2 (truth-ambiguous). If $T_{\text{src}} = \mathsf{F}$, Eq. (3) only implies that at least one of $\mathcal{T}[S_1], \mathcal{T}[S_2]$ is F, but does not determine which one(s). In this case, Step I uses $L_i = \mathsf{Attr}(L_{\text{src}}, S_i)$ as evidence and only outputs branch PUs with $L_i \neq \emptyset$, assigning them $T_i = \mathsf{F}$.

To justify this inference, apply Eq. (10) (the F branch):

$$L_{\text{src}} = \mathcal{L}[S_{\text{src}}] = \bigcup_{S_j \in \{S_1, S_2\}\ \wedge\ \mathcal{T}[S_j] = \mathsf{F}} \mathcal{L}[S_j].$$

Thus, if a branch S_i is *true*, its link set $\mathcal{L}[S_i]$ is not included in the union and contributes no bindings to L_{src} over $\mathsf{Vars}(S_i)$, implying $\mathsf{Attr}(L_{\text{src}}, S_i) = \emptyset$. Conversely, if $\mathsf{Attr}(L_{\text{src}}, S_i) \neq \emptyset$, then necessarily $\mathcal{T}[S_i] = \mathsf{F}$ and (by the same projection argument as above) $\mathsf{Attr}(L_{\text{src}}, S_i) = \mathcal{L}[S_i]$. Therefore, every branch PU that Step I actually outputs in this case has tag $\langle T_i, L_i \rangle = \langle \mathsf{F}, \mathcal{L}[S_i] \rangle = \langle \mathcal{T}[S_i], \mathcal{L}[S_i] \rangle$ and is precise by Definition 1. (If $L_i = \emptyset$, Step I discards the branch PU; there is nothing to prove.)

More formally, let $V_i = \mathsf{Vars}(S_i)$. Under $T_{\text{src}} = \mathsf{F}$, Eq. (10) can be rewritten as

$$L_{\text{src}} = \bigcup_{j \in J} \mathcal{L}[S_j], \quad \text{where } J = \{\, j \in \{1, 2\} \mid \mathcal{T}[S_j] = \mathsf{F} \,\}.$$

If $\mathcal{T}[S_i] = \mathsf{T}$, then $i \notin J$ and every link $l \in L_{\text{src}}$ belongs to $\mathcal{L}[S_j]$ for some $j \neq i$. With uniquely named bound variables, $\mathsf{Vars}(S_i) \cap \mathsf{Vars}(S_j) = \emptyset$, hence $l \restriction_{V_i} = \emptyset$ for all such l and thus $\mathsf{Attr}(L_{\text{src}}, S_i) = \emptyset$.

Conversely, if $\mathsf{Attr}(L_{\text{src}}, S_i) \neq \emptyset$, then by Definition 4 there exists some $l \in L_{\text{src}}$ with $l \restriction_{V_i} \neq \emptyset$. This cannot happen when $l \in \mathcal{L}[S_j]$ for $j \neq i$ (their restrictions are empty), therefore $l \in \mathcal{L}[S_i]$ and $i \in J$, implying $\mathcal{T}[S_i] = \mathsf{F}$. Moreover, since links from other branches restrict to $\emptyset$ on V_i, Definition 4 yields $\mathsf{Attr}(L_{\text{src}}, S_i) = \mathcal{L}[S_i]$.

This completes the proof for connective-rooted decomposition for and. The same reasoning applies to or and implies by using their truth/link rules: whenever Step I assigns a branch truth by Boolean determinacy, it matches $\mathcal{T}[S_i]$; whenever Step I infers a branch truth from $L_i \neq \emptyset$, the corresponding link generation rule guarantees that $L_i \neq \emptyset$ holds exactly for contributing branches and that $L_i = \mathcal{L}[S_i]$.

(2) Decomposition-based extraction for quantifiers

Assume S_{src} is rooted at a quantifier $Q \in \{\forall, \exists\}$, i.e., $S_{\text{src}} = Qx \in C\,(s)$, where C is the quantifier domain (a context set) provided by P_{src}. Step I removes the quantifier and outputs an open PU $u = \langle s, P_{\text{src}}, \langle x, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle$ with $C_{\mathsf{T}}, C_{\mathsf{F}}$ derived from L_{src} using Extract.

By the quantifier link rules, $L_{\text{src}} = \mathcal{L}[Qx \in C(s)]$ contains exactly the *critical* bindings of x: satisfying bindings for $\exists$ (Eq. (9)) and violating bindings for $\forall$ (Eq. (8)). Therefore, define

$$E = \mathsf{Extract}(L_{\text{src}}, x).$$

We distinguish subcases by the source truth value T_{src} (as in our methodology). Note that $\exists$ generates links only when satisfied (Eq. (9)), whereas $\forall$ generates links only when violated (Eq. (8)).

Case 1 ($T_{\text{src}} = \mathsf{T}$). If $L_{\text{src}} \neq \emptyset$, then necessarily $Q = \exists$ and Eq. (9) with Definition 5 yields

$$E = \{\, e \in C \mid \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T} \,\}.$$

Step I sets $C_{\mathsf{T}} = E$ and $C_{\mathsf{F}} = C \setminus E$. Then $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$, and by the characterization of E we have $\forall e \in C_{\mathsf{T}}$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{F}}$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$, hence Definition 2 holds. If instead $L_{\text{src}} = \emptyset$, then necessarily $Q = \forall$ and the universal constraint is satisfied; by Eq. (1) we have $\forall e \in C$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$. Step I can set $C_{\mathsf{T}} = C$ and $C_{\mathsf{F}} = \emptyset$, which also satisfies Definition 2.

Case 2 ($T_{\text{src}} = \mathsf{F}$). If $L_{\text{src}} \neq \emptyset$, then necessarily $Q = \forall$ and Eq. (8) with Definition 5 yields

$$E = \{\, e \in C \mid \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F} \,\}.$$

Step I sets $C_{\mathsf{F}} = E$ and $C_{\mathsf{T}} = C \setminus E$. Then $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$ and Definition 2 holds by the characterization of E. If instead $L_{\text{src}} = \emptyset$, then necessarily $Q = \exists$ and the existential constraint is violated; by Eq. (2) we have $\forall e \in C$, $\mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$. Step I can set $C_{\mathsf{T}} = \emptyset$ and $C_{\mathsf{F}} = C$, which again satisfies Definition 2.

(3) Preservation-based extraction

Step I may also preserve the whole source constraint as a closed PU $u = \langle S_{\text{src}}, P_{\text{src}}, \langle T_{\text{src}}, L_{\text{src}} \rangle \rangle$. Since $\langle T_{\text{src}}, L_{\text{src}} \rangle = \langle \mathcal{T}[S_{\text{src}}], \mathcal{L}[S_{\text{src}}] \rangle$ for a correct CC implementation, this PU is precise by Definition 1.

Moreover, Step I may preserve S_{src} as a degenerate open PU by introducing a fresh dummy variable m such that $m \notin \mathsf{Vars}(S_{\text{src}})$, together with a universal context C_{U} (the set of all possible elements). The constructed PU is

$$u_{\text{dummy}} = \langle S_{\text{src}}, P_{\text{src}}, \langle m, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle,$$

where $C_{\mathsf{T}} = C_{\mathsf{U}}$ and $C_{\mathsf{F}} = \emptyset$ if $T_{\text{src}} = \mathsf{T}$, and $C_{\mathsf{T}} = \emptyset$ and $C_{\mathsf{F}} = C_{\mathsf{U}}$ if $T_{\text{src}} = \mathsf{F}$. Since m does not syntactically occur in S_{src}, binding m cannot affect the evaluation of S_{src}, i.e., for any $e \in C_{\mathsf{U}}$ we have $\mathcal{T}[S_{\text{src}}]_{\{m \mapsto e\}} = \mathcal{T}[S_{\text{src}}] = T_{\text{src}}$. Therefore, the above choice of $C_{\mathsf{T}}, C_{\mathsf{F}}$ satisfies Definition 2, and u_{dummy} is precise.

Combining the above cases, every PU that Step I outputs is either a precise closed unit or a precise open unit, hence precise by Definition 3. □

B. Soundness of Step II

Step II expands the PU repository by (i) syntactically composing existing PUs using Boolean operators (not, and, or, implies), and (ii) deriving the new semantic tag by directly mirroring CC's semantics (Sec. I). Therefore, if the dependent PU tags are already precise, the derived tag remains precise.

Lemma 2 (Soundness of Step II)

Given precise PU(s), any new complex PU constructed via CCM's syntactic composition and tag derivation rules inherently remains precise.

Proof. Consider any single application of Step II that takes precise PU(s) from the repository and produces a new PU $u_{\text{new}} = \langle s_{\text{new}}, p_{\text{new}}, t_{\text{new}} \rangle$. Step II may first perform preprocessing (systematic renaming of variables and context names) to avoid name clashes. This renaming is capture-avoiding and applied consistently to the constraint and the tag; hence it preserves the values of $\mathcal{T}[\cdot]$ and $\mathcal{L}[\cdot]$ up to alpha-equivalence. Thus, it suffices to prove soundness for the core composition and tag-derivation rules.

We distinguish whether u_{new} is closed or open.

(1) Composing closed PUs

Assume the dependent PUs are closed and precise: $u = \langle s, p, \langle T, L \rangle \rangle$ and/or $u_i = \langle s_i, p_i, \langle T_i, L_i \rangle \rangle$ (Definitions 1 and 3). We show that the derived tag equals CC semantics for s_{new} under p_{new}, hence Definition 1 holds.

Unary not**.** Step II forms $s_{\text{new}} = \mathsf{not}(s)$ and sets $t_{\text{new}} = \langle \neg T, L \rangle$. Since u is precise, $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$. By Eqs. (6) and (13), $\mathcal{T}[\mathsf{not}(s)] = \neg \mathcal{T}[s]$ and $\mathcal{L}[\mathsf{not}(s)] = \mathcal{L}[s]$. Therefore $t_{\text{new}} = \langle \mathcal{T}[s_{\text{new}}], \mathcal{L}[s_{\text{new}}] \rangle$, i.e., u_{new} is precise by Definition 1.

Binary and**.** Step II forms $s_{\text{new}} = (s_1)$ and (s_2), sets $p_{\text{new}} = p_1 \cup p_2$, and derives $T' = T_1 \wedge T_2$ and $L' = L_1 \otimes L_2$ if $T' = \mathsf{T}$, otherwise $L' = \bigcup \{ L_i \mid T_i = \mathsf{F} \}$. Because

u_1, u_2 are precise, $T_i = \mathcal{T}[s_i]$ and $L_i = \mathcal{L}[s_i]$. By Eq. (3), we have $T' = \mathcal{T}[(s_1)$ and $(s_2)]$. Moreover, Eq. (10) defines exactly the same case split on the operand truth values (equivalently on T'), yielding $L' = \mathcal{L}[(s_1)$ and $(s_2)]$. Hence $t_{\text{new}} = \langle \mathcal{T}[s_{\text{new}}], \mathcal{L}[s_{\text{new}}] \rangle$ and u_{new} is precise.

Binary or**.** This case is analogous to and by using Eqs. (4) and (11): Step II derives $T' = T_1 \vee T_2$ and sets L' to the union of the true-branch link sets when $T' = \mathsf{T}$, otherwise to $L_1 \otimes L_2$; substituting $T_i = \mathcal{T}[s_i]$ and $L_i = \mathcal{L}[s_i]$ yields exactly $\langle \mathcal{T}[s_{\text{new}}], \mathcal{L}[s_{\text{new}}] \rangle$.

Binary implies**.** Step II derives implication by the structural equivalence (s_1) implies $(s_2) \equiv (\mathsf{not}(s_1))$ or (s_2). By the already established soundness of unary not and binary or, the derived tag matches the CC semantics of $(\mathsf{not}(s_1))$ or (s_2), which coincides with Eqs. (5) and (12). Therefore, the composed closed PU is precise.

(2) Composing open PUs

Assume the dependent PUs are open and precise: $u_1 = \langle s_1, p_1, \langle x, C_{\mathsf{T}1}, C_{\mathsf{F}1} \rangle \rangle$ and $u_2 = \langle s_2, p_2, \langle y, C_{\mathsf{T}2}, C_{\mathsf{F}2} \rangle \rangle$ (Definitions 2 and 3), where each s_i has at most one free variable. Step II unifies the free variables by a fresh z and uses capture-avoiding substitution, yielding $s_1' = s_1[z/x]$ and $s_2' = s_2[z/y]$, so the composed constraint has at most one free variable (z). We show that the derived $\langle z, C_{\mathsf{T}}', C_{\mathsf{F}}' \rangle$ satisfies Definition 2.

Unary not**.** For $u = \langle s, p, \langle x, C_{\mathsf{T}}, C_{\mathsf{F}} \rangle \rangle$, Step II sets $u_{\text{new}} = \langle \mathsf{not}(s), p, \langle x, C_{\mathsf{F}}, C_{\mathsf{T}} \rangle \rangle$. Since u is precise, $C_{\mathsf{T}} \cap C_{\mathsf{F}} = \emptyset$ and $\forall e \in C_{\mathsf{T}},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{F}},\ \mathcal{T}[s]_{\{x \mapsto e\}} = \mathsf{F}$. By Eq. (6), this immediately implies $\forall e \in C_{\mathsf{F}},\ \mathcal{T}[\mathsf{not}(s)]_{\{x \mapsto e\}} = \mathsf{T}$ and $\forall e \in C_{\mathsf{T}},\ \mathcal{T}[\mathsf{not}(s)]_{\{x \mapsto e\}} = \mathsf{F}$, so Definition 2 holds.

Binary and**.** Step II forms $s_{\text{new}} = (s_1')$ and (s_2'), sets $p_{\text{new}} = p_1 \cup p_2$, and derives $C_{\mathsf{T}}' = C_{\mathsf{T}1} \cap C_{\mathsf{T}2}$ and $C_{\mathsf{F}}' = C_{\mathsf{F}1} \cup C_{\mathsf{F}2}$. Disjointness holds because if $e \in C_{\mathsf{T}}'$, then $e \in C_{\mathsf{T}1}$ and $e \in C_{\mathsf{T}2}$, hence $e \notin C_{\mathsf{F}1}$ and $e \notin C_{\mathsf{F}2}$ (Definition 2 for u_1, u_2), so $e \notin C_{\mathsf{F}}'$. For truth correctness, take any $e \in C_{\mathsf{T}}'$; then $\mathcal{T}[s_1']_{\{z \mapsto e\}} = \mathsf{T}$ and $\mathcal{T}[s_2']_{\{z \mapsto e\}} = \mathsf{T}$ by Definition 2 for u_1, u_2 and the consistent unification. By Eq. (3), $\mathcal{T}[s_{\text{new}}]_{\{z \mapsto e\}} = \mathsf{T}$. Similarly, for any $e \in C_{\mathsf{F}}'$, at least one of the two sub-constraints is false under $\{z \mapsto e\}$, so Eq. (3) yields $\mathcal{T}[s_{\text{new}}]_{\{z \mapsto e\}} = \mathsf{F}$. Therefore, Definition 2 holds for the new open PU.

Binary or **and** implies**.** These cases follow the same pattern as and: Step II's derived contexts are chosen to mirror the Boolean truth conditions. For or, it sets $C_{\mathsf{T}}' = C_{\mathsf{T}1} \cup C_{\mathsf{T}2}$ and $C_{\mathsf{F}}' = C_{\mathsf{F}1} \cap C_{\mathsf{F}2}$, matching Eq. (4). For implies, it sets $C_{\mathsf{T}}' = C_{\mathsf{F}1} \cup C_{\mathsf{T}2}$ and $C_{\mathsf{F}}' = C_{\mathsf{T}1} \cap C_{\mathsf{F}2}$, matching Eq. (5). In both cases, disjointness and the required satisfaction/violation conditions follow immediately from Definition 2 for the dependent PUs and the corresponding Boolean equation.

Combining the above, every Step II rule application maps precise inputs to a precise output (Definitions 1–3). □

C. Soundness of Step III

Step III takes a selected precise PU from the repository and constructs a follow-up test case (P_f, S_f) and an assertion A. For a closed PU, Step III may apply a randomness injection to enlarge certain context sets in P_f to increase diversity. For an open PU, Step III closes it by binding a quantifier and a target context using a semantic forcing strategy so that the expected outcome is guaranteed. We now show that, under a correct CC implementation, the follow-up execution always satisfies the derived assertion.

Lemma 3 (Soundness of Step III)

Given a precise PU u selected by Step III, let (P_f, S_f) and assertion A be the follow-up test case constructed from u. For a correct CC implementation, if $I_{CC}(P_f, S_f) = \langle T_f, L_f \rangle$, then A holds on $\langle T_f, L_f \rangle$.

Proof. We consider two categories depending on whether u is closed or open.

Auxiliary notations

For a link l (a set of bindings), define $\mathsf{Vars}(l) = \{ v \mid \exists e,\ (v \mapsto e) \in l \}$. For a link set L, define $\mathsf{Vars}(L) = \bigcup_{l \in L} \mathsf{Vars}(l)$. We also define a link-set extension relation: for two link sets L' and L,

$$L' \sqsupseteq L \overset{\text{def}}{\iff} \forall l \in L,\ \exists l' \in L' \text{ s.t. } l \subseteq l'.$$

Intuitively, $L' \sqsupseteq L$ means every original critical binding pattern is preserved (possibly with extra bindings) after pool enlargement.

(1) The selected PU is closed

Let the selected precise closed PU be $u = \langle s, p, \langle T, L \rangle \rangle$. By Definition 1, for a correct CC implementation we have $I_{CC}(p, s) = \langle T, L \rangle$, i.e., $T = \mathcal{T}[s]$ and $L = \mathcal{L}[s]$ under pool p. Step III constructs the follow-up constraint as $S_f = s$ and initializes the follow-up pool as $P_f = p$.

If no randomness injection is applied, then $I_{CC}(P_f, S_f) = I_{CC}(p, s) = \langle T, L \rangle$. The derived assertion for this case requires the follow-up truth to match ($T_f = T$), and allows the follow-up link set to be an extension of the original one. Since equality implies extension, $L_f = L$ satisfies $L_f \sqsupseteq L$.

Now consider the case where Step III applies randomness injection to obtain an enlarged pool p^+ from p. Following our methodology, the injection only adds random elements to the contexts corresponding to variables in $\mathsf{Vars}(L)$, and keeps all other contexts unchanged. Let $I_{CC}(p^+, s) = \langle T^+, L^+ \rangle$. We show $T^+ = T$ and $L^+ \sqsupseteq L$, which implies the assertion holds.

We prove the following structural claim by induction on the syntax of s:

To make the implicit dependency on the pool explicit, we write $\mathcal{T}_p[s]_\alpha$ and $\mathcal{L}_p[s]_\alpha$ for evaluating s under pool p with an (already-applied) assignment set α.

Claim. For any assignment set α, let p^+ be obtained from p by enlarging only the context sets of variables in $\mathsf{Vars}(\mathcal{L}_p[s]_\alpha)$ (computed under p). Then the truth value is unchanged and the new link set extends the old one, i.e., $\mathcal{T}_{p^+}[s]_\alpha = \mathcal{T}_p[s]_\alpha$ and $\mathcal{L}_{p^+}[s]_\alpha \sqsupseteq \mathcal{L}_p[s]_\alpha$.

Base cases

If s is a bfunction, then by Eq. (14) we have $\mathcal{L}_p[s]_\alpha = \emptyset$ for all p, α. Thus $\mathsf{Vars}(\mathcal{L}_p[s]_\alpha) = \emptyset$ and no injection happens; the claim is trivial.

Inductive steps

If $s = \mathsf{not}(s_0)$, then by Eqs. (6) and (13) the truth is negated and the links are preserved; more precisely, $\mathcal{T}_p[\mathsf{not}(s_0)]_\alpha = \neg\mathcal{T}_p[s_0]_\alpha$ and $\mathcal{L}_p[\mathsf{not}(s_0)]_\alpha = \mathcal{L}_p[s_0]_\alpha$. Therefore $\mathsf{Vars}(\mathcal{L}_p[\mathsf{not}(s_0)]_\alpha) = \mathsf{Vars}(\mathcal{L}_p[s_0]_\alpha)$, and the induction hypothesis on s_0 yields the claim for s.

If s is composed by a connective (and, or, or implies), then the truth of each operand is preserved by the induction hypothesis. Let $s = (s_1)\ \mathsf{op}\ (s_2)$ where $\mathsf{op} \in \{\mathsf{and}, \mathsf{or}, \mathsf{implies}\}$. By the induction hypothesis applied to s_1 and s_2, we have $\mathcal{T}_{p^+}[s_i]_\alpha = \mathcal{T}_p[s_i]_\alpha$ for $i \in \{1, 2\}$. Hence the Boolean condition that selects the semantic branch of the link-generation rule is unchanged. Concretely, in Eqs. (10)–(12), the predicates such as $\mathcal{T}[s_1]_\alpha = \mathsf{T}$ and $\mathcal{T}[s_2]_\alpha = \mathsf{F}$ hold under p if and only if they hold under p^+. Therefore, the same constructor is used to compute links before and after injection (i.e., a Cartesian product $\otimes$ in the concurrent case, or a union/selection of operand link sets in the alternative case).

It remains to show that these constructors preserve the extension relation.

Union preserves extension. If $A' \sqsupseteq A$ and $B' \sqsupseteq B$, then $A' \cup B' \sqsupseteq A \cup B$. Indeed, take any $l \in A \cup B$. If $l \in A$, pick $l' \in A'$ with $l \subseteq l'$; then $l' \in A' \cup B'$. The case $l \in B$ is symmetric.

Cartesian product preserves extension. If $A' \sqsupseteq A$ and $B' \sqsupseteq B$, then $A' \otimes B' \sqsupseteq A \otimes B$. If $A = \emptyset$ or $B = \emptyset$, then by the definition of $\otimes$ we have $A \otimes B = A \cup B$ and $A' \otimes B' = A' \cup B'$, and the result follows from the union argument above. Otherwise, take any $l \in A \otimes B$; then $l = l_1 \cup l_2$ for some $l_1 \in A$ and $l_2 \in B$. Choose $l_1' \in A'$ with $l_1 \subseteq l_1'$ and $l_2' \in B'$ with $l_2 \subseteq l_2'$. Then $l' = l_1' \cup l_2' \in A' \otimes B'$ and $l \subseteq l'$.

Combining the unchanged truth-branch selection with the above two preservation facts, we obtain $\mathcal{L}_{p^+}[s]_\alpha \sqsupseteq \mathcal{L}_p[s]_\alpha$ for all connectives.

If s is rooted at a quantifier $Qx \in C(s_0)$ with $Q \in \{\forall, \exists\}$, we consider two subcases. If $\mathcal{L}_p[Qx \in C(s_0)]_\alpha = \emptyset$, then $\mathsf{Vars}(\mathcal{L}_p[Qx \in C(s_0)]_\alpha) = \emptyset$ and thus this quantifier node triggers no injection for its domain C. Moreover, since no pool component used by evaluating $Qx \in C(s_0)$ is modified, for every $e \in C$ the induction hypothesis on s_0 ensures $\mathcal{T}_{p^+}[s_0]_{\alpha\cup\{x\mapsto e\}} = \mathcal{T}_p[s_0]_{\alpha\cup\{x\mapsto e\}}$ and $\mathcal{L}_{p^+}[s_0]_{\alpha\cup\{x\mapsto e\}} = \mathcal{L}_p[s_0]_{\alpha\cup\{x\mapsto e\}}$. Therefore, Eqs. (1)–(9) yield $\mathcal{T}_{p^+}[Qx \in C(s_0)]_\alpha = \mathcal{T}_p[Qx \in C(s_0)]_\alpha$ and $\mathcal{L}_{p^+}[Qx \in C(s_0)]_\alpha = \mathcal{L}_p[Qx \in C(s_0)]_\alpha = \emptyset$, and the claim holds. Otherwise, by Eqs. (8) and (9), non-emptiness of links implies that $Q = \exists$ with overall truth T, or $Q = \forall$ with overall truth F. Enlarging C cannot flip these outcomes: an existential constraint remains true as long as an existing witness remains in the domain, and a universal constraint remains false as long as an existing counterexample remains in the domain. Hence, $\mathcal{T}_{p^+}[Qx \in C(s_0)]_\alpha = \mathcal{T}_p[Qx \in C(s_0)]_\alpha$ still holds.

Because injection only adds elements and never removes them, every original critical element $e \in C$ that contributed to $\mathcal{L}_p[Qx \in C(s_0)]_\alpha$ remains in the enlarged domain $C^+ \supseteq C$. Moreover, every original link in $\mathcal{L}_p[Qx \in C(s_0)]_\alpha$ is generated by $\{\{x \mapsto e\}\} \otimes l_0$ for some such critical e and some $l_0 \in \mathcal{L}_p[s_0]_{\alpha\cup\{x\mapsto e\}}$ (by Eqs. (8)–(9)). By the induction hypothesis on s_0 under the same binding $\alpha \cup \{x \mapsto e\}$, the new inner link set contains some $l_0' \in \mathcal{L}_{p^+}[s_0]_{\alpha\cup\{x\mapsto e\}}$ with $l_0 \subseteq l_0'$. Therefore, the enlarged quantifier link set contains $\{\{x \mapsto e\}\} \otimes l_0'$, and we have $\{\{x \mapsto e\}\} \otimes l_0 \subseteq \{\{x \mapsto e\}\} \otimes l_0'$. This shows that every original link is preserved up to extension. Finally, enlarging C may introduce additional critical elements $e' \in C^+ \setminus C$ (e.g., new witnesses for $\exists$ or new counterexamples for $\forall$), which can only add extra links and does not invalidate the extension relation. Hence $\mathcal{L}_{p^+}[Qx \in C(s_0)]_\alpha \sqsupseteq \mathcal{L}_p[Qx \in C(s_0)]_\alpha$.

This completes the induction and proves the claim, so $T^+ = T$ and $L^+ \sqsupseteq L$. Therefore, the derived closed-PU assertion $\big(T_f = T \ \wedge\ L_f \sqsupseteq L\big)$ holds.

(2) The selected PU is open

Let the selected precise open PU be $u = \langle s_o, p_o, \langle m, C_\mathsf{T}, C_\mathsf{F}\rangle\rangle$, where s_o has at most one free variable m. By Definition 2, $C_\mathsf{T} \cap C_\mathsf{F} = \emptyset$, and $\forall e \in C_\mathsf{T},\ \mathcal{T}[s_o]_{\{m\mapsto e\}} = \mathsf{T}$ and $\forall e \in C_\mathsf{F},\ \mathcal{T}[s_o]_{\{m\mapsto e\}} = \mathsf{F}$.

Step III closes u by constructing a follow-up constraint $S_f = Qm \in C_\mathsf{target}(s_o)$ and a follow-up pool $P_f = p_o \cup \{C_\mathsf{target}\}$, where $Q \in \{\forall, \exists\}$ and C_target is chosen by semantic forcing. We show the follow-up execution satisfies the assertion in all forcing subcases.

Goal: forcing satisfaction ($T_f = \mathsf{T}$)

If Step III chooses $Q = \forall$, it enforces $C_\mathsf{target} \subseteq C_\mathsf{T}$ and applies no randomness injection. Then by Eq. (1), $\mathcal{T}[S_f] = \mathsf{T}$. Moreover, since $\forall$ generates links only when violated (Eq. (8)), we have $\mathcal{L}[S_f] = \emptyset$. Thus the assertion $\big(T_f = \mathsf{T} \ \wedge\ L_f = \emptyset\big)$ holds.

If Step III chooses $Q = \exists$, it enforces $C_\mathsf{target} \cap C_\mathsf{T} \neq \emptyset$ (and may inject random elements into C_target as noise). Pick any $e \in C_\mathsf{target} \cap C_\mathsf{T}$; then $\mathcal{T}[s_o]_{\{m\mapsto e\}} = \mathsf{T}$, so Eq. (2) yields $\mathcal{T}[S_f] = \mathsf{T}$. By Eq. (9), for every satisfying element in the domain, the link set contains a link that binds m to that element; hence $\mathsf{Extract}(L_f, m) \supseteq (C_\mathsf{target} \cap C_\mathsf{T})$. Therefore the assertion $\big(T_f = \mathsf{T} \ \wedge\ \mathsf{Extract}(L_f, m) \supseteq (C_\mathsf{target} \cap C_\mathsf{T})\big)$ holds.

Goal: forcing violation ($T_f = \mathsf{F}$)

If Step III chooses $Q = \exists$, it enforces $C_\mathsf{target} \subseteq C_\mathsf{F}$ and applies no randomness injection. Then all elements in the domain violate s_o, so Eq. (2) yields $\mathcal{T}[S_f] = \mathsf{F}$. Since $\exists$ generates links only when satisfied (Eq. (9)), we have $\mathcal{L}[S_f] = \emptyset$. Thus the assertion $\big(T_f = \mathsf{F} \ \wedge\ L_f = \emptyset\big)$ holds.

If Step III chooses $Q = \forall$, it enforces $C_\mathsf{target} \cap C_\mathsf{F} \neq \emptyset$ (and may inject random elements into C_target). Pick any $e \in C_\mathsf{target} \cap C_\mathsf{F}$; then $\mathcal{T}[s_o]_{\{m\mapsto e\}} = \mathsf{F}$, so Eq. (1) yields $\mathcal{T}[S_f] = \mathsf{F}$. By Eq. (8), every violating element contributes a link binding m to it, hence $\mathsf{Extract}(L_f, m) \supseteq (C_\mathsf{target} \cap C_\mathsf{F})$.

Therefore the assertion $\big(T_f = \mathsf{F} \ \wedge \ \mathsf{Extract}(L_f, m) \supseteq (C_{\mathsf{target}} \cap C_{\mathsf{F}})\big)$ holds.

In all cases, the follow-up execution result of a correct CC implementation satisfies the derived assertion. □

IV. Proof of the Soundness Theorem

Theorem (Soundness of CCM)

Any correct CC implementation satisfies all test cases and assertions generated by CCM.

Proof. Consider an arbitrary run of CCM over any source test case $(P_{\mathrm{src}}, S_{\mathrm{src}})$. Assume the CC implementation under test is correct, i.e., for any input (P, S) it returns $I_{CC}(P, S) = \langle \mathcal{T}[S], \mathcal{L}[S] \rangle$ under the semantics in Sec. I.

In Step I, CCM executes the source test case and extracts a set of initial PUs from the source execution result. By Lemma 1, every PU that Step I outputs is precise (Definition 3).

In Step II, CCM repeatedly composes existing PUs and derives tags to expand the PU repository. By Lemma 2, each single rule application maps precise input PU(s) to a precise output PU. Therefore, by induction on the number of Step II compositions performed in the run, every PU in the repository after Step II is precise.

In Step III, CCM selects a PU from the repository and constructs a follow-up test case (P_f, S_f) and an assertion A. Since the selected PU is precise, Lemma 3 guarantees that the follow-up execution result $I_{CC}(P_f, S_f) = \langle T_f, L_f \rangle$ satisfies the derived assertion A.

Because the above argument holds for every source test case processed by CCM and for every follow-up test case and assertion it generates, any correct CC implementation satisfies all test cases and assertions generated by CCM. □